

# Lesson 01 Demo 02

## Implementing PASTA Model

**Objective:** To implement the PASTA (Process for Attack Simulation and Threat Analysis) model for assessing risks and recommending counter measures

**Tools required:** Python

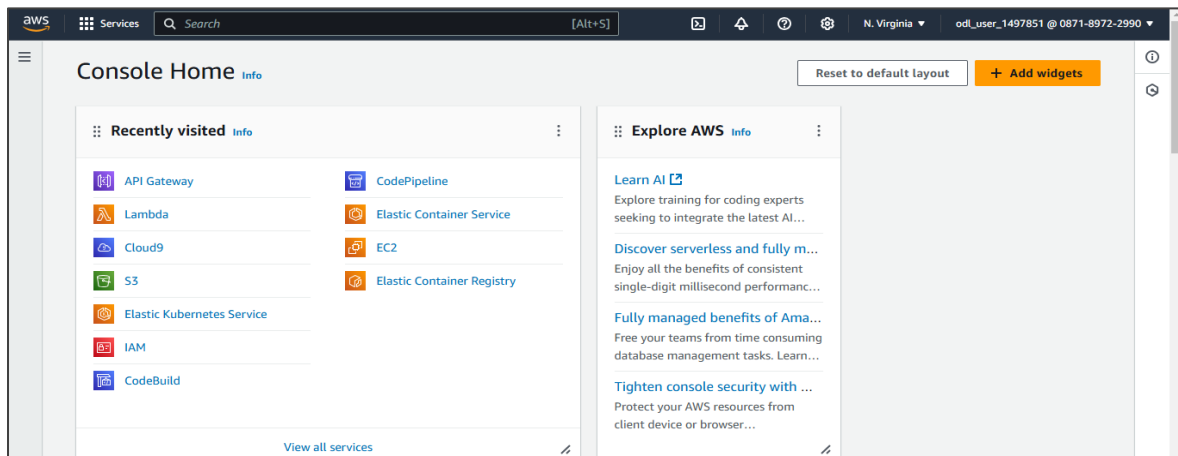
**Prerequisites:** None

Steps to be followed:

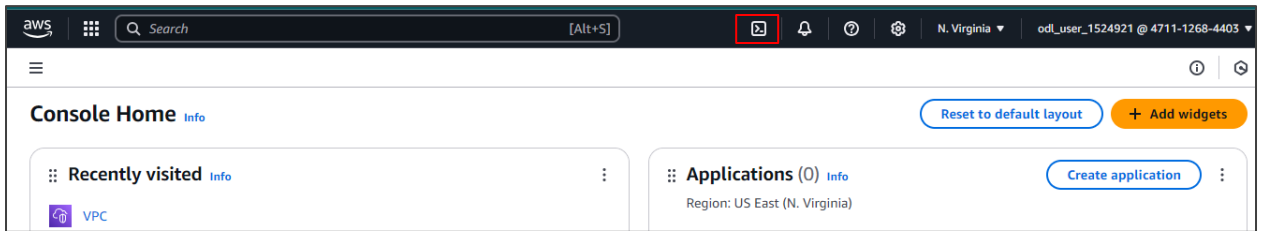
1. Implement the PASTA model using a Python script

### Step 1: Implement the PASTA model using a Python script

- 1.1 Open the **AWS Management Console** in your practice environment



1.2 Click on the cloud shell icon for the terminal



1.3 Run the following command to verify if Python is installed:

**python3 --version**

```
[cloudshell-user@ip-10-130-76-22 ~]$ python3 --version
Python 3.9.16
[cloudshell-user@ip-10-130-76-22 ~]$
```

1.4 Execute the commands given below to create and navigate to the directory:

**mkdir python**

**cd python**

```
Python 3.9.16
[cloudshell-user@ip-10-130-76-22 ~]$ mkdir python
[cloudshell-user@ip-10-130-76-22 ~]$ cd python
[cloudshell-user@ip-10-130-76-22 python]$
```

1.5 Run the following command to create a file:

**nano pasta\_model.py**

```
[cloudshell-user@ip-10-130-76-22 python]$ nano pasta_model.py
[cloudshell-user@ip-10-130-76-22 python]$
```

1.6 Once the file opens, add the code given below, save it, and then exit:

```
class PASTA:
    def __init__(self, system_info, objectives, threats, vulnerabilities, impact,
countermeasures):
        # System and objectives definition
        self.system_info = system_info
        self.objectives = objectives
        self.threats = threats
        self.vulnerabilities = vulnerabilities
        self.impact = impact
        self.countermeasures = countermeasures

    def stage_1_define_objectives(self):
        # Stage 1: Define objectives of the security process
        print(f"System: {self.system_info}")
        print(f"Objectives: {self.objectives}\n")

    def stage_2_threat_landscape_analysis(self):
        # Stage 2: Threat Landscape Analysis
        print("Identified Threats:")
        for threat in self.threats:
            print(f"- {threat}")
        print("\n")

    def stage_3_vulnerability_analysis(self):
        # Stage 3: Vulnerability Analysis
        print("Identified Vulnerabilities:")
        for vuln in self.vulnerabilities:
            print(f"- {vuln}")
        print("\n")

    def stage_4_attack_simulation(self):
        # Stage 4: Attack Simulation (Simple Risk Simulation based on impact and
likelihood)
        risk_scores = {}
        for threat in self.threats:
            risk = self.impact.get(threat, 0) * self.vulnerabilities.get(threat, 0)
            risk_scores[threat] = risk

        print("Risk Assessment based on Threats and Vulnerabilities:")
        for threat, risk in risk_scores.items():
            print(f"Threat: {threat}, Risk Score: {risk}")
        print("\n")
```

```

def stage_5_risk_impact_analysis(self):
    # Stage 5: Risk and Impact Analysis
    total_risk = sum([self.impact.get(threat, 0) * self.vulnerabilities.get(threat, 0) for
threat in self.threats])
    print(f"Total Risk Score: {total_risk}")

def stage_6_countermeasure_recommendation(self):
    # Stage 6: Suggest countermeasures based on vulnerabilities
    print("Recommended Countermeasures:")
    for vuln in self.vulnerabilities:
        print(f"- {self.countermeasures.get(vuln, 'No countermeasure available')}")
    print("\n")

def stage_7_monitoring_and_improvement(self):
    # Stage 7: Continuous Monitoring (Simulation)
    print("Implementing continuous monitoring...")
    # In real-world scenarios, this might involve monitoring logs, attacks, and
updates.
    print("Monitoring system for new threats and vulnerabilities.\n")

# Example of system data and PASTA parameters
def main():
    system_info = "Web Application Server"
    objectives = "Ensure confidentiality, integrity, and availability of user data."

    threats = ["SQL Injection", "Cross-Site Scripting (XSS)", "Denial of Service (DoS)"]
    vulnerabilities = {
        "SQL Injection": 8,
        "Cross-Site Scripting (XSS)": 6,
        "Denial of Service (DoS)": 7
    }

    impact = {
        "SQL Injection": 9,
        "Cross-Site Scripting (XSS)": 5,
        "Denial of Service (DoS)": 7
    }

    countermeasures = {
        "SQL Injection": "Use parameterized queries and ORM.",
        "Cross-Site Scripting (XSS)": "Sanitize and validate user input.",
        "Denial of Service (DoS)": "Use rate limiting and DDOS protection."
    }

```

```
# Instantiate PASTA model
pasta_model = PASTA(system_info, objectives, threats, vulnerabilities, impact,
countermeasures)
```

```
# Run through the stages of the PASTA model
pasta_model.stage_1_define_objectives()
pasta_model.stage_2_threat_landscape_analysis()
pasta_model.stage_3_vulnerability_analysis()
pasta_model.stage_4_attack_simulation()
pasta_model.stage_5_risk_impact_analysis()
pasta_model.stage_6_countermeasure_recommendation()
pasta_model.stage_7_monitoring_and_improvement()
```

```
if __name__ == "__main__":
    main()
```

```
GNU nano 5.8 pasta_model.py
    "SQL Injection": "Use parameterized queries and ORM.",
    "Cross-Site Scripting (XSS)": "Sanitize and validate user input.",
    "Denial of Service (DoS)": "Use rate limiting and DDOS protection."
}

# Instantiate PASTA model
pasta_model = PASTA(system_info, objectives, threats, vulnerabilities, impact, countermeasures)

# Run through the stages of the PASTA model
pasta_model.stage_1_define_objectives()
pasta_model.stage_2_threat_landscape_analysis()
pasta_model.stage_3_vulnerability_analysis()
pasta_model.stage_4_attack_simulation()
pasta_model.stage_5_risk_impact_analysis()
pasta_model.stage_6_countermeasure_recommendation()
pasta_model.stage_7_monitoring_and_improvement()

if __name__ == "__main__":
    main()
```

**Note:** To save and exit the script, use CTRL + O >> press enter >> CTRL + X

This script offers a practical implementation of the PASTA model. It simulates all seven stages, providing a hands-on approach to understanding and applying threat analysis frameworks in cybersecurity.

1.7 Once the Python script is created, run it using the following command:  
**python3 pasta\_model.py**

```
[cloudshell-user@ip-10-130-76-22 python]$ python3 pasta_model.py
System: Web Application Server
Objectives: Ensure confidentiality, integrity, and availability of user data.

Identified Threats:
- SQL Injection
- Cross-Site Scripting (XSS)
- Denial of Service (DoS)

Identified Vulnerabilities:
- SQL Injection
- Cross-Site Scripting (XSS)
- Denial of Service (DoS)

Risk Assessment based on Threats and Vulnerabilities:
Threat: SQL Injection, Risk Score: 72
Threat: Cross-Site Scripting (XSS), Risk Score: 30
Threat: Denial of Service (DoS), Risk Score: 49

Total Risk Score: 151
Recommended Countermeasures:
- Use parameterized queries and ORM.
```

By following these steps, you have successfully implemented the PASTA model to assess risks and recommend countermeasures.